Day 16 - React에서 Next.js로

> **Phase 4: Next.js** | 학습일: 16일차

왜 필요한가?

지금까지 우리는 React로 할일 앱을 만들었습니다. React만으로도 멋진 앱을 만들 수 있지만, 실제 서비스를 운영하려면 몇 가지 문제에 부딪힙니다.

- **검색엔진이 우리 사이트를 못 읽는다** (SEO 문제)
- **첫 화면이 나올 때까지 시간이 오래 걸린다** (빈 화면 깜빡임)
- **라우팅, 이미지 최적화, API 서버 등을 별도로 설정해야 한다**

Next.js는 이런 문제들을 한 번에 해결해주는 React 기반 프레임워크입니다.

실생활 비유

React를 **자동차**라고 생각해보세요. 엔진(컴포넌트 시스템)은 훌륭하지만, 네비게이션(라우팅), 에어백(에러 처리), 크루즈 컨트롤(자동 최적화) 같은 편의 장치는 직접 달아야 합니다.

Next.js는 이 자동차에 **네비게이션, 에어백, 크루즈 컨트롤, 블랙박스까지 기본 장착한 완성차**입니다. 엔진은 같은 React 엔진을 사용하되, 나머지를 알아서 챙겨줍니다.

---

1. Next.js란?

Next.js는 Vercel이라는 회사가 만든 **React 기반 풀스택 웹 프레임워크**입니다.

React vs Next.js 비교

구분	React (CRA)	Next.js
역할	UI 라이브러리	풀스택 프레임워크
렌더링	CSR만 가능	SSR + CSR + SSG 모두 가능
라우팅	react-router 별도 설치	파일 기반 라우팅 내장
SEO	불리 (빈 HTML)	유리 (완성된 HTML)
API 서버	별도 백엔드 필요	Route Handlers 내장
이미지 최적화	직접 구현	next/image 내장
배포	별도 설정 필요	Vercel 원클릭 배포

Before (React만 사용할 때)

```
# 라이브러리를 하나하나 설치해야 했다
npm install react-router-dom    # 라우팅
npm install express              # API 서버
npm install react-helmet         # SEO 메타태그
```

After (Next.js를 사용하면)

```
# 다 내장되어 있다!
npx create-next-app@latest my-app
# 라우팅, API, SEO, 이미지 최적화... 전부 포함
```

---

2. SSR vs CSR 차이

이것은 Next.js를 이해하는 데 가장 중요한 개념입니다.

실생활 비유

CSR (Client Side Rendering) = 밀키트 배달

1. 밀키트(재료 = JavaScript 파일)가 집(브라우저)에 도착한다
2. 집에서 직접 요리한다 (브라우저가 JavaScript를 실행해서 화면을 그린다)

3. 요리가 완성될 때까지 빈 접시만 보인다 (흰 화면)

SSR (Server Side Rendering) = 음식점에서 완성된 음식 서빙

1. 주방(서버)에서 요리를 완성한다 (서버가 HTML을 완성한다)
2. 완성된 요리(완성된 HTML)가 테이블(브라우저)에 나온다
3. 바로 먹을 수 있다 (바로 화면이 보인다)

기술적 비교

구분	CSR (Client Side Rendering)	SSR (Server Side Rendering)
렌더링 위치	브라우저 (클라이언트)	서버
초기 로딩	HTML이 비어있음 (JS 다운로드 후 렌더링)	완성된 HTML이 바로 전달됨
SEO	불리함 (검색엔진이 빈 HTML만 봄)	유리함 (검색엔진이 완성된 HTML을 봄)
사용자 경험	초기에 빈 화면 → JS 로드 후 표시	바로 콘텐츠가 보임
서버 부하	낮음 (브라우저가 일함)	높음 (서버가 일함)
대표 예시	React (CRA), Vue (CLI)	Next.js, Nuxt.js

SEO 관점에서의 중요성

검색엔진(구글, 네이버)의 크롤러(로봇)는 웹사이트를 방문해서 HTML을 읽습니다.

```html
<!— CSR: 검색엔진 로봇이 보는 것 -->
<html>
  <body>
    <div id="root"></div>  <!— 아무 내용도 없음! -->
    <script src="bundle.js"></script>
  </body>
</html>

<!— SSR: 검색엔진 로봇이 보는 것 -->
<html>
  <body>
    <h1>할일 관리 앱</h1>
    <ul>
      <li>TypeScript 공부하기</li>  <!— 내용이 다 있음! -->
      <li>Next.js 배우기</li>
    </ul>
  </body>
</html>
```

검색엔진은 JavaScript를 실행하지 않기 때문에, CSR 사이트는 빈 페이지로 인식합니다. 반면 SSR 사이트는 완성된 HTML을 읽을 수 있어 검색 결과에 잘 노출됩니다.

---

3. create-next-app으로 프로젝트 생성

```
npx create-next-app@latest my-todo-app
```

설치 중 물어보는 질문들:

```
Would you like to use TypeScript? → Yes
Would you like to use ESLint? → Yes
Would you like to use Tailwind CSS? → No (일단 기본 CSS 사용)
Would you like to use `src/` directory? → Yes
Would you like to use App Router? → Yes (중요!)
Would you like to use Turbopack for next dev? → Yes
Would you like to customize the import alias? → No
```

```
cd my-todo-app
npm run dev
```

브라우저에서 `http://localhost:3000` 접속하면 Next.js 시작 페이지가 나타납니다.

4. App Router 구조

실생활 비유: 아파트 구조

- `layout.tsx` = **아파트 건물의 공용 공간** (복도, 엘리베이터, 로비). 모든 세대가 공유한다.
- `page.tsx` = **각 세대(호실)**. 세대마다 다른 내용이 들어있다.
- `globals.css` = **아파트 관리 규정** (건물 전체에 적용되는 스타일 규칙).

프로젝트 구조

```
src/app/
  layout.tsx     ← 전체 레이아웃 (HTML, body 태그 포함, 공용 복도)
  page.tsx       ← "/" 경로의 페이지 (101호)
  globals.css    ← 전역 스타일 (관리 규정)
```

layout.tsx 기본 구조

```typescript
// src/app/layout.tsx
import type { Metadata } from 'next';
import './globals.css';

// SEO를 위한 메타데이터
export const metadata: Metadata = {
  title: '할일 관리 앱',
  description: 'Next.js로 만든 할일 관리 앱',
};

// children은 page.tsx의 내용이 들어오는 자리
export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="ko">
      <body>{children}</body>
    </html>
  );
}
```

page.tsx 기본 구조

```
// src/app/page.tsx
export default function Home() {
  return (
    <main>
      <h1>할일 관리 앱</h1>
      <p>Next.js로 만든 첫 번째 페이지입니다.</p>
    </main>
  );
}
```

> **핵심 포인트**: `layout.tsx` 의 `{children}` 자리에 `page.tsx` 의 내용이 끼워집니다. 마치 액자(layout) 안에 그림(page)을 넣는 것과 같습니다.

---

5. 'use client' 지시어

서버 컴포넌트가 기본인 이유

Next.js 13 이후(App Router)에서는 모든 컴포넌트가 기본적으로 **서버 컴포넌트**입니다.

왜 그럴까요?

1. **번들 크기 감소**: 서버에서 실행되는 코드는 브라우저에 전송되지 않습니다
2. **보안**: API 키나 데이터베이스 접근 코드가 브라우저에 노출되지 않습니다
3. **성능**: 서버에서 미리 렌더링하면 사용자가 더 빨리 화면을 볼 수 있습니다

언제 'use client'를 붙이는가?

다음 기능을 사용할 때 파일 맨 위에 `'use client'` 를 추가합니다:

기능	예시	'use client' 필요?
useState	상태 관리	필요
useEffect	사이드 이펙트	필요
onClick, onChange 등	이벤트 핸들러	필요
usePathname, useRouter	클라이언트 라우팅 훅	필요
단순 표시용 컴포넌트	텍스트, 이미지 표시	불필요
async 데이터 패칭	서버에서 데이터 가져오기	불필요

```
'use client'; // 파일 맨 첫 줄에 작성

import { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      클릭 횟수: {count}
    </button>
  );
}
```

자주 하는 실수

```
// 실수 1: 'use client' 없이 useState 사용
import { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0); // 에러 발생!
  // "You're importing a component that needs useState.
  //  It only works in a Client Component..."
  return <button>{count}</button>;
}

// 해결: 파일 맨 위에 'use client' 추가
```

```
// 실수 2: 'use client'를 import 아래에 작성
import { useState } from 'react';
'use client'; // 에러! import보다 위에 있어야 합니다

// 해결: 파일의 첫 번째 줄에 작성
'use client';
import { useState } from 'react';
```

---

6. 파일 기반 라우팅

React에서는 react-router-dom을 설치하고 라우트를 코드로 정의했습니다. Next.js에서는 **폴더를 만들면 자동으로 URL이 생깁니다**.

Before (React Router)

```
// React에서의 라우팅 설정
import { BrowserRouter, Routes, Route } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/todos" element={<Todos />} />
      </Routes>
    </BrowserRouter>
  );
}
```

After (Next.js 파일 기반 라우팅)

```
src/app/
  page.tsx            →  /
  about/page.tsx      →  /about
  todos/page.tsx      →  /todos
```

코드를 한 줄도 쓰지 않아도 폴더 구조만으로 라우팅이 완성됩니다!

파일 경로	URL	설명
`src/app/page.tsx`	`/`	메인 페이지
`src/app/about/page.tsx`	`/about`	소개 페이지
`src/app/todos/page.tsx`	`/todos`	할일 목록 페이지
`src/app/todos/[id]/page.tsx`	`/todos/1`, `/todos/2`	할일 상세 (동적 라우트)

Day 16 정리

개념	핵심 요약
Next.js	React 기반 풀스택 프레임워크 (SSR + CSR + API)
SSR	서버에서 완성된 HTML 전달 (SEO 유리, 빠른 첫 화면)
CSR	브라우저에서 JavaScript로 화면 생성 (기존 React 방식)
App Router	`app/` 폴더 기반 라우팅 시스템
layout.tsx	모든 페이지를 감싸는 공용 레이아웃
page.tsx	URL 경로에 대응하는 페이지 컴포넌트
'use client'	클라이언트 컴포넌트 지정 (useState, onClick 등 사용 시 필수)

Day 17 - 라우팅과 레이아웃

> **Phase 4: Next.js** | 학습일: 17일차

왜 필요한가?

실제 웹사이트는 한 페이지로 끝나지 않습니다. 홈, 할일 목록, 완료 목록, 통계 등 **여러 페이지**가 있고, 이 페이지들 사이를 **매끄럽게 이동**할 수 있어야 합니다. 또한 네비게이션 바(Navbar)처럼 **모든 페이지에 공통으로 보이는 부분**도 있어야 합니다.

Day 17에서는 Next.js의 강력한 라우팅 시스템과 레이아웃 중첩 기능을 배웁니다.

---

1. 폴더 기반 라우팅

핵심 원리: 폴더 구조 = URL 구조

Next.js App Router에서는 `app/` 폴더 안에 폴더를 만들고 그 안에 `page.tsx`를 넣으면 자동으로 해당 URL이 생깁니다.

```
src/app/
├── layout.tsx          ← 전체 레이아웃 (모든 페이지 공통)
├── page.tsx            ← "/" 경로
├── todos/
│   └── page.tsx        ← "/todos" 경로
├── completed/
│   └── page.tsx        ← "/completed" 경로
└── stats/
    └── page.tsx        ← "/stats" 경로
```

실생활 비유

컴퓨터의 폴더 구조를 생각해보세요:

```
내 문서/
├── 보고서/
│   └── 월간보고.hwp
├── 사진/
│   └── 여행사진.jpg
└── 음악/
    └── 좋아하는노래.mp3
```

탐색기에서 `내 문서 > 보고서 > 월간보고.hwp` 로 찾아가듯이, Next.js에서도 `app > todos > page.tsx` 가 `/todos` URL이 됩니다.

---

2. Link 컴포넌트

`<a>` 태그 vs `<Link>` 컴포넌트

구분	`<a>` 태그	`<Link>` 컴포넌트
페이지 이동 시	전체 페이지 새로고침	필요한 부분만 교체 (SPA 방식)
속도	느림 (모든 리소스 재로딩)	빠름 (변경된 부분만 로드)
사용자 경험	화면이 깜빡임	부드러운 전환
상태 유지	모든 상태 초기화	공통 레이아웃 상태 유지

실생활 비유

- `<a>` 태그로 이동 = **집을 나갔다가 다른 집으로 이사**. 짐을 다 싸서 옮겨야 합니다.
- `<Link>` 로 이동 = **같은 건물 안에서 다른 방으로 이동**. 복도(레이아웃)는 그대로고 방(페이지)만 바뀝니다.

사용법

```
import Link from 'next/link';

export default function Home() {
  return (
    <nav>
      {/* a 태그 대신 Link 사용 */}
      <Link href="/">홈</Link>
      <Link href="/todos">할일 목록</Link>
      <Link href="/completed">완료 목록</Link>
      <Link href="/stats">통계</Link>
    </nav>
  );
}
```

prefetch 동작

Link 컴포넌트는 **화면에 보이는 순간** 해당 페이지의 데이터를 미리 가져옵니다(prefetch). 그래서 사용자가 링크를 클릭했을 때 거의 즉시 페이지가 표시됩니다.

```
{/* prefetch 기본값은 true (자동으로 미리 로드) */}
<Link href="/todos">할일 목록</Link>

{/* prefetch를 끄고 싶을 때 */}
<Link href="/todos" prefetch={false}>할일 목록</Link>
```

---

3. usePathname()으로 현재 경로 감지

네비게이션 바에서 **현재 페이지에 해당하는 메뉴를 강조 표시**하려면 현재 URL 경로를 알아야 합니다.

```
'use client'; // usePathname은 클라이언트 훅이므로 필수!

import { usePathname } from 'next/navigation';
import Link from 'next/link';

export default function Navbar() {
  const pathname = usePathname(); // 현재 URL 경로 (예: "/todos")

  const links = [
    { href: '/', label: '홈' },
    { href: '/todos', label: '할일' },
    { href: '/completed', label: '완료' },
    { href: '/stats', label: '통계' },
  ];

  return (
    <nav>
      {links.map((link) => (
        <Link
          key={link.href}
          href={link.href}
          style={{
            fontWeight: pathname === link.href ? 'bold' : 'normal',
            color: pathname === link.href ? '#0070f3' : '#333',
          }}
        >
          {link.label}
        </Link>
      ))}
    </nav>
  );
}
```

자주 하는 실수

```
// 실수: 'use client' 없이 usePathname 사용
import { usePathname } from 'next/navigation';

export default function Navbar() {
  const pathname = usePathname(); // 에러!
  // usePathname은 클라이언트 훅이므로 'use client'가 필요합니다
}
```

---

4. 레이아웃 중첩

실생활 비유: 러시아 인형 (마트료시카)

레이아웃 중첩은 마트료시카 인형과 같습니다. 큰 인형(루트 레이아웃) 안에 중간 인형(섹션 레이아웃)이 있고, 그 안에 작은 인형(페이지)이 있습니다.

```
RootLayout (app/layout.tsx)          ← 가장 큰 인형 (Navbar + Footer)
  └── TodosLayout (app/todos/layout.tsx)  ← 중간 인형 (사이드바)
        └── TodosPage (app/todos/page.tsx)   ← 가장 작은 인형 (할일 목록)
```

구현 예시

```tsx
// src/app/layout.tsx (루트 레이아웃 - 모든 페이지에 적용)
import Navbar from '@/components/Navbar';

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="ko">
      <body>
        <Navbar />            {/* 모든 페이지에 Navbar 표시 */}
        <main>{children}</main>
        <footer>2026 할일 앱</footer>
      </body>
    </html>
  );
}
```

```tsx
// src/app/todos/layout.tsx (할일 섹션 전용 레이아웃)
export default function TodosLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <div style={{ display: 'flex' }}>
      <aside>
        <h3>할일 메뉴</h3>
        <ul>
          <li>전체 보기</li>
          <li>중요한 것</li>
        </ul>
      </aside>
      <section>{children}</section>  {/* 할일 페이지 내용 */}
    </div>
  );
}
```

레이아웃 중첩 결과

`/todos` 페이지에 접속하면:

```
┌──────────────────────────────────┐
│ Navbar (RootLayout)              │  ← 루트 레이아웃
├────────┬─────────────────────────┤
│ 할일   │                         │
│ 메뉴   │  할일 목록 페이지 내용  │  ← TodosLayout + TodosPage
│        │                         │
├────────┴─────────────────────────┤
│ Footer (RootLayout)              │  ← 루트 레이아웃
└──────────────────────────────────┘
```

> **핵심 포인트**: 페이지를 이동할 때 레이아웃은 다시 렌더링되지 않습니다. `page.tsx` 부분만 교체됩니다. 이것이 Next.js가 빠른 이유 중 하나입니다.

---

5. Navbar 구현 (활성 메뉴 표시)

완성된 Navbar 컴포넌트:

```
// src/components/Navbar.tsx
'use client';

import Link from 'next/link';
import { usePathname } from 'next/navigation';

const navLinks = [
  { href: '/', label: '홈' },
  { href: '/todos', label: '할일 목록' },
  { href: '/completed', label: '완료 목록' },
  { href: '/stats', label: '통계' },
];

export default function Navbar() {
  const pathname = usePathname();

  return (
    <nav style={{
      display: 'flex',
      gap: '16px',
      padding: '16px',
      backgroundColor: '#f5f5f5',
      borderBottom: '1px solid #ddd',
    }}>
      {navLinks.map((link) => {
        const isActive = pathname === link.href;

        return (
          <Link
            key={link.href}
            href={link.href}
            style={{
              padding: '8px 16px',
              borderRadius: '4px',
              textDecoration: 'none',
              backgroundColor: isActive ? '#0070f3' : 'transparent',
              color: isActive ? 'white' : '#333',
              fontWeight: isActive ? 'bold' : 'normal',
            }}
          >
            {link.label}
          </Link>
        );
      })}
    </nav>
  );
}
```

Day 17 정리

개념	핵심 요약
폴더 기반 라우팅	`app/todos/page.tsx` = `/todos` URL
Link 컴포넌트	`<Link href="/todos">` - 페이지 리로드 없이 이동 (SPA)
prefetch	Link가 화면에 보이면 자동으로 대상 페이지를 미리 로드
usePathname	현재 URL 경로를 가져오는 클라이언트 훅
레이아웃 중첩	각 폴더의 layout.tsx가 마트료시카처럼 겹겹이 감싸는 구조
Navbar	Link + usePathname으로 현재 메뉴 강조 표시

Day 18 - 서버와 클라이언트 컴포넌트

> **Phase 4: Next.js** | 학습일: 18일차

왜 필요한가?

Next.js의 가장 강력한 특징은 **서버 컴포넌트와 클라이언트 컴포넌트를 함께 사용할 수 있다는 것**입니다. 이 두 가지를 적절히 나누면:

- **번들 크기가 줄어들어** 사이트가 빨라집니다
- **보안이 강화**됩니다 (API 키가 브라우저에 노출되지 않음)
- **데이터 패칭이 간단**해집니다 (async/await 직접 사용)

실생활 비유: 주방(서버)과 홀(클라이언트)

음식점을 상상해보세요:

- **주방(서버 컴포넌트)**: 식재료 준비, 요리, 위생 관리 등 고객에게 보이지 않는 작업을 합니다. 고객은 주방에 들어갈 수 없습니다.
- **홀(클라이언트 컴포넌트)**: 고객이 메뉴를 고르고, 주문 버튼을 누르고, 음식을 받아 먹는 공간입니다. 고객과의 상호작용이 일어납니다.

같은 원리입니다:

- **서버 컴포넌트** = 데이터를 가져오고 준비하는 작업 (사용자에게 보이지 않음)
- **클라이언트 컴포넌트** = 사용자와 상호작용하는 UI (버튼 클릭, 입력 등)

---

1. Server Component vs Client Component 상세 비교

구분	Server Component	Client Component
기본값	기본 (아무 지시어 없으면 서버)	`'use client'` 선언 필요
실행 위치	서버에서만 실행	서버 + 브라우저 양쪽에서 실행
React 훅	사용 불가 (useState, useEffect 등)	사용 가능
이벤트 핸들러	사용 불가 (onClick, onChange 등)	사용 가능
async/await	컴포넌트 함수에서 직접 사용 가능	useEffect 안에서만 가능
번들 크기	클라이언트 JS에 포함 안됨 (가벼움)	클라이언트 JS에 포함됨
보안	API 키, DB 접근 코드 노출 안됨	브라우저에서 코드가 보임
데이터 패칭	직접 DB 접근, fetch 가능	fetch API로 서버에 요청해야 함
적합한 용도	정적 콘텐츠, 데이터 표시	폼, 버튼, 상태 관리

2. async 서버 컴포넌트에서 데이터 패칭

서버 컴포넌트의 가장 큰 장점은 **컴포넌트 자체를 async로 만들 수 있다는 것**입니다.

Before (React에서의 데이터 패칭)

```
// React: useEffect + useState로 복잡하게 처리
'use client';
import { useState, useEffect } from 'react';

export default function TodoList() {
  const [todos, setTodos] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetch('/api/todos')
      .then((res) => res.json())
      .then((data) => setTodos(data))
      .catch((err) => setError(err))
      .finally(() => setLoading(false));
  }, []);

  if (loading) return <p>로딩 중...</p>;
  if (error) return <p>에러 발생!</p>;

  return (
    <ul>
      {todos.map((todo) => (
        <li key={todo.id}>{todo.title}</li>
      ))}
    </ul>
  );
}
```

After (Next.js 서버 컴포넌트)

```
// Next.js: async/await으로 간단하게 처리
import { getTodos } from '@/lib/todos';

export default async function TodoList() {
  const todos = await getTodos(); // 직접 데이터 가져오기!

  return (
    <ul>
      {todos.map((todo) => (
        <li key={todo.id}>{todo.title}</li>
      ))}
    </ul>
  );
}
```

useEffect, useState, loading 상태, error 상태를 전부 관리할 필요가 없습니다!

3. 데이터 전달 패턴 (서버 -> props -> 클라이언트)

서버 컴포넌트에서 데이터를 가져와서, props로 클라이언트 컴포넌트에 전달하는 것이 권장 패턴입니다.

```
서버 컴포넌트 (page.tsx) ──데이터 패칭──→ 데이터 획득
        │
        │ props로 전달
        ▼
클라이언트 컴포넌트 (TodoApp.tsx) ──→ 사용자와 상호작용
```

구현 예시

```
// src/app/page.tsx (서버 컴포넌트 - 데이터 패칭 담당)
import { getTodos } from '@/lib/todos';
import TodoApp from '@/components/TodoApp';

export default async function Home() {
  const initialTodos = await getTodos(); // 서버에서 데이터 가져오기

  return (
    <main>
      <h1>할일 관리</h1>
      {/* props로 클라이언트 컴포넌트에 전달 */}
      <TodoApp initialTodos={initialTodos} />
    </main>
  );
}
```

```tsx
// src/components/TodoApp.tsx (클라이언트 컴포넌트 - 상호작용 담당)
'use client';

import { useState } from 'react';
import { Todo } from '@/types/todo';

interface TodoAppProps {
  initialTodos: Todo[]; // 서버에서 받은 초기 데이터
}

export default function TodoApp({ initialTodos }: TodoAppProps) {
  // 서버에서 받은 데이터를 초기 상태로 사용
  const [todos, setTodos] = useState<Todo[]>(initialTodos);
  const [input, setInput] = useState('');

  const addTodo = () => {
    if (!input.trim()) return;
    const newTodo: Todo = {
      id: Date.now(),
      title: input,
      completed: false,
    };
    setTodos([...todos, newTodo]);
    setInput('');
  };

  return (
    <div>
      <div>
        <input
          value={input}
          onChange={(e) => setInput(e.target.value)}
          placeholder="할일을 입력하세요"
        />
        <button onClick={addTodo}>추가</button>
      </div>
      <ul>
        {todos.map((todo) => (
          <li key={todo.id}>{todo.title}</li>
        ))}
      </ul>
    </div>
  );
}
```

4. 컴포넌트 분리 전략

판단 기준표

이 기능이 필요한가?	서버 컴포넌트	클라이언트 컴포넌트
데이터를 가져와서 표시만	O	
버튼 클릭, 폼 입력		O
useState, useEffect		O
async/await 데이터 패칭	O	
API 키, DB 접근	O	
SEO가 중요한 콘텐츠	O	
사용자와 실시간 상호작용		O

실전 분리 예시

```
src/
├── app/
│   └── page.tsx              ← 서버 컴포넌트 (데이터 패칭)
├── components/
│   ├── TodoApp.tsx           ← 클라이언트 ('use client')
│   ├── TodoForm.tsx          ← 클라이언트 ('use client')
│   ├── TodoItem.tsx          ← 클라이언트 ('use client')
│   ├── TodoStats.tsx         ← 서버 (표시만 하므로)
│   └── Header.tsx            ← 서버 (표시만 하므로)
├── lib/
│   └── todos.ts              ← 서버 전용 로직
└── types/
    └── todo.ts               ← 공용 타입 정의
```

자주 하는 실수

```
// 실수: 서버 컴포넌트 안에서 클라이언트 기능 사용
export default function Page() {
  const [count, setCount] = useState(0); // 에러!
  // 서버 컴포넌트에서는 useState를 사용할 수 없습니다
}

// 해결: 상호작용이 필요한 부분만 클라이언트 컴포넌트로 분리
// page.tsx (서버)
import Counter from '@/components/Counter';

export default function Page() {
  return <Counter />; // 클라이언트 컴포넌트 사용
}

// components/Counter.tsx (클라이언트)
'use client';
import { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}
```

> **핵심 원칙**: `'use client'` 경계를 최대한 아래로 내리세요. 전체 페이지를 클라이언트로 만들지 말고, 상호작용이 필요한 작은 컴포넌트만 클라이언트로 만드세요.

---

5. lib/ 폴더에 서버 로직 분리

`lib/` 폴더는 서버에서만 실행되는 데이터 관련 코드를 모아두는 곳입니다.

```typescript
// src/lib/todos.ts
import { Todo } from '@/types/todo';

// 임시 데이터 저장소 (나중에 데이터베이스로 교체)
let todos: Todo[] = [
  { id: 1, title: 'Next.js 배우기', completed: false },
  { id: 2, title: 'TypeScript 복습', completed: true },
  { id: 3, title: '포트폴리오 만들기', completed: false },
];

// 전체 할일 가져오기
export async function getTodos(): Promise<Todo[]> {
  return todos;
}

// ID로 할일 하나 가져오기
export async function getTodoById(id: number): Promise<Todo | undefined> {
  return todos.find((todo) => todo.id === id);
}

// 새 할일 추가
export async function addTodo(title: string): Promise<Todo> {
  const newTodo: Todo = {
    id: Date.now(),
    title,
    completed: false,
  };
  todos.push(newTodo);
  return newTodo;
}
```

```typescript
// src/types/todo.ts
export interface Todo {
  id: number;
  title: string;
  completed: boolean;
}
```

Day 18 정리

개념	핵심 요약
Server Component	기본값, async 가능, 훅 불가, 번들에 미포함
Client Component	`'use client'` 필요, 훅/이벤트 사용 가능
데이터 전달 패턴	서버에서 패칭 -> props로 클라이언트에 전달
분리 전략	`'use client'` 경계를 최대한 아래(작은 단위)로
lib/ 폴더	서버에서만 실행되는 데이터 로직 모음
types/ 폴더	서버/클라이언트 공용 타입 정의

Day 19 - Next.js 세부 기능

> **Phase 4: Next.js** | 학습일: 19일차

왜 필요한가?

기본적인 페이지를 만들 수 있게 되었으니, 이제 **실제 서비스 수준의 기능**을 추가할 차례입니다:

- **SEO 최적화**: 검색엔진에 잘 노출되도록 메타데이터를 설정
- **로딩 UI**: 데이터를 불러오는 동안 사용자에게 로딩 상태를 보여주기
- **에러 처리**: 오류가 발생했을 때 앱이 죽지 않고 적절한 안내를 보여주기
- **404 페이지**: 존재하지 않는 페이지에 접근했을 때 안내하기
- **이미지 최적화**: 이미지를 자동으로 최적화하여 빠르게 로딩하기

실생활 비유

음식점으로 비유하면:

- **Metadata** = 가게 간판과 메뉴판 (지나가는 사람이 어떤 가게인지 알 수 있음 = 검색엔진)
- **loading.tsx** = "잠시만 기다려주세요" 안내판 (요리 중일 때)
- **error.tsx** = "죄송합니다, 재료가 소진되었습니다" 안내 (문제 발생 시)
- **not-found.tsx** = "해당 메뉴는 없습니다" 안내 (잘못된 주문 시)

---

1. Metadata API (SEO)

Metadata는 HTML의 `<head>` 태그 안에 들어가는 정보입니다. 검색엔진과 SNS에서 사이트를 표시할 때 사용합니다.

정적 메타데이터

```
// src/app/layout.tsx
import type { Metadata } from 'next';

export const metadata: Metadata = {
  title: '할일 관리 앱',
  description: 'Next.js로 만든 할일 관리 앱입니다',
};
```

이렇게 하면 HTML에 자동으로 다음이 추가됩니다:

```
<head>
  <title>할일 관리 앱</title>
  <meta name="description" content="Next.js로 만든 할일 관리 앱입니다" />
</head>
```

title 템플릿

루트 레이아웃에서 title 템플릿을 설정하면, 하위 페이지의 제목이 자동으로 조합됩니다.

```
// src/app/layout.tsx
export const metadata: Metadata = {
  title: {
    template: '%s | 할일 앱',  // %s 자리에 하위 페이지 제목이 들어감
    default: '할일 앱',          // 하위 페이지에 title이 없을 때 기본값
  },
  description: 'Next.js 할일 관리 앱',
};
```

```
// src/app/todos/page.tsx
export const metadata: Metadata = {
  title: '할일 목록',  // 결과: "할일 목록 | 할일 앱"
};
```

```
// src/app/stats/page.tsx
export const metadata: Metadata = {
  title: '통계',  // 결과: "통계 | 할일 앱"
};
```

generateMetadata (동적 메타데이터)

할일 상세 페이지처럼 URL에 따라 제목이 달라져야 할 때 사용합니다.

```
// src/app/todos/[id]/page.tsx
import type { Metadata } from 'next';
import { getTodoById } from '@/lib/todos';

// 동적 메타데이터 생성 함수
export async function generateMetadata({
  params,
}: {
  params: Promise<{ id: string }>;
}): Promise<Metadata> {
  const { id } = await params;
  const todo = await getTodoById(Number(id));

  return {
    title: todo ? todo.title : '할일을 찾을 수 없습니다',
    // /todos/1 → "Next.js 배우기 | 할일 앱"
    // /todos/999 → "할일을 찾을 수 없습니다 | 할일 앱"
  };
}
```

---

2. loading.tsx (로딩 UI)

`loading.tsx` 파일을 만들면 해당 경로의 페이지가 로딩되는 동안 **자동으로** 로딩 UI가 표시됩니다. React Suspense를 기반으로 동작합니다.

기본 사용법

```react-ts
// src/app/loading.tsx (전역 로딩 UI)
export default function Loading() {
  return (
    <div style={{
      display: 'flex',
      justifyContent: 'center',
      alignItems: 'center',
      minHeight: '200px',
    }}>
      <div>
        <p>로딩 중입니다...</p>
        <div style={{
          width: '40px',
          height: '40px',
          border: '4px solid #f3f3f3',
          borderTop: '4px solid #0070f3',
          borderRadius: '50%',
          animation: 'spin 1s linear infinite',
        }} />
      </div>
    </div>
  );
}
```

경로별 로딩 UI

각 폴더에 별도의 `loading.tsx`를 넣으면 해당 경로 전용 로딩 UI를 보여줄 수 있습니다.

```
src/app/
  loading.tsx          ← 전역 로딩 UI
  todos/
    loading.tsx        ← /todos 전용 로딩 UI
    [id]/
      loading.tsx      ← /todos/[id] 전용 로딩 UI
```

동작 원리

```
사용자가 /todos 접속
       |
       ▼
loading.tsx가 먼저 표시됨
       |
       ▼ (page.tsx의 데이터 패칭 완료)
       |
       ▼
page.tsx 내용으로 교체됨
```

> **핵심 포인트**: `loading.tsx` 를 만들어두기만 하면 Next.js가 알아서 로딩/완료 전환을 처리합니다. 별도의 loading state를 관리할 필요가 없습니다.

---

3. error.tsx (에러 처리)

`error.tsx` 는 해당 경로에서 에러가 발생했을 때 보여주는 페이지입니다.

중요: 반드시 'use client'를 붙여야 합니다

에러 처리 UI는 클라이언트에서 동작해야 하므로 `'use client'` 가 필수입니다.

```tsx
// src/app/error.tsx
'use client'; // 필수!

export default function Error({
  error,
  reset,
}: {
  error: Error & { digest?: string };
  reset: () => void;
}) {
  return (
    <div style={{
      padding: '40px',
      textAlign: 'center',
    }}>
      <h2>오류가 발생했습니다</h2>
      <p style={{ color: '#666' }}>{error.message}</p>
      <button
        onClick={reset}  // 다시 시도 (페이지 리렌더링)
        style={{
          padding: '8px 16px',
          backgroundColor: '#0070f3',
          color: 'white',
          border: 'none',
          borderRadius: '4px',
          cursor: 'pointer',
          marginTop: '16px',
        }}
      >
        다시 시도
      </button>
    </div>
  );
}
```

error.tsx의 두 가지 props

prop	타입	설명
`error`	`Error`	발생한 에러 객체 (message 속성으로 에러 내용 확인)
`reset`	`() => void`	호출하면 에러가 발생한 영역을 다시 렌더링 시도

자주 하는 실수

```
// 실수: 'use client' 빼먹기
export default function Error({ error, reset }) {
  // 에러! error.tsx는 반드시 'use client'가 필요합니다
  return <div>{error.message}</div>;
}
```

---

4. not-found.tsx (404 페이지)

존재하지 않는 페이지에 접근했을 때 보여주는 커스텀 404 페이지입니다.

기본 not-found.tsx

```
// src/app/not-found.tsx
import Link from 'next/link';

export default function NotFound() {
  return (
    <div style={{
      padding: '40px',
      textAlign: 'center',
    }}>
      <h2>페이지를 찾을 수 없습니다</h2>
      <p>요청하신 페이지가 존재하지 않습니다.</p>
      <Link
        href="/"
        style={{
          display: 'inline-block',
          marginTop: '16px',
          padding: '8px 16px',
          backgroundColor: '#0070f3',
          color: 'white',
          borderRadius: '4px',
          textDecoration: 'none',
        }}
      >
        홈으로 돌아가기
      </Link>
    </div>
  );
}
```

notFound() 함수

코드에서 프로그래밍 방식으로 404 페이지를 표시할 수 있습니다.

```
// src/app/todos/[id]/page.tsx
import { notFound } from 'next/navigation';
import { getTodoById } from '@/lib/todos';

export default async function TodoDetailPage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  const { id } = await params;
  const todo = await getTodoById(Number(id));

  // 할일이 존재하지 않으면 404 페이지 표시
  if (!todo) {
    notFound(); // not-found.tsx가 렌더링됨
  }

  return (
    <div>
      <h1>{todo.title}</h1>
      <p>상태: {todo.completed ? '완료' : '미완료'}</p>
    </div>
  );
}
```

---

5. 동적 라우트 [id]

URL에 변하는 값(파라미터)이 포함된 경우 대괄호(`[]`)를 사용합니다.

폴더 구조

```
src/app/todos/
  page.tsx          ← /todos (할일 목록)
  [id]/
    page.tsx        ← /todos/1, /todos/2, /todos/3 ... (할일 상세)
```

구현 예시

```tsx
// src/app/todos/[id]/page.tsx
import { getTodoById } from '@/lib/todos';
import { notFound } from 'next/navigation';
import Link from 'next/link';

export default async function TodoDetailPage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  const { id } = await params;
  const todo = await getTodoById(Number(id));

  if (!todo) {
    notFound();
  }

  return (
    <div style={{ padding: '20px' }}>
      <Link href="/todos">← 목록으로 돌아가기</Link>
      <h1>{todo.title}</h1>
      <p>
        상태:{' '}
        <span style={{
          color: todo.completed ? 'green' : 'orange'
        }}>
          {todo.completed ? '완료' : '진행 중'}
        </span>
      </p>
      <p>ID: {todo.id}</p>
    </div>
  );
}
```

할일 목록에서 상세 페이지로 링크

```tsx
// src/app/todos/page.tsx
import { getTodos } from '@/lib/todos';
import Link from 'next/link';

export default async function TodosPage() {
  const todos = await getTodos();

  return (
    <div>
      <h1>할일 목록</h1>
      <ul>
        {todos.map((todo) => (
          <li key={todo.id}>
            <Link href={`/todos/${todo.id}`}>
              {todo.title}
            </Link>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

---

6. next/image 이미지 최적화

Next.js의 `<Image>` 컴포넌트는 이미지를 자동으로 최적화합니다.

`<img>` 태그 vs `<Image>` 컴포넌트

구분	`<img>` 태그	Next.js `<Image>`
이미지 크기	원본 그대로 전송	디바이스에 맞게 자동 리사이즈
포맷	원본 포맷 (PNG, JPG)	WebP/AVIF 자동 변환 (더 작은 용량)
로딩	한꺼번에 로드	Lazy Loading (화면에 보일 때 로드)
레이아웃 시프트	이미지 로드 시 레이아웃이 밀림	자동으로 공간 확보 (밀림 방지)

사용법

```
import Image from 'next/image';

export default function Profile() {
  return (
    <div>
      {/* 로컬 이미지 */}
      <Image
        src="/images/profile.png"  // public 폴더 기준
        alt="프로필 사진"
        width={200}
        height={200}
      />

      {/* 외부 이미지 (next.config.ts에 도메인 등록 필요) */}
      <Image
        src="https://example.com/photo.jpg"
        alt="외부 사진"
        width={400}
        height={300}
      />
    </div>
  );
}
```

외부 이미지 사용 시 설정

```
// next.config.ts
import type { NextConfig } from 'next';

const nextConfig: NextConfig = {
  images: {
    remotePatterns: [
      {
        protocol: 'https',
        hostname: 'example.com',
      },
    ],
  },
};

export default nextConfig;
```

자주 하는 실수

```
// 실수: width, height를 지정하지 않음
<Image src="/photo.jpg" alt="사진" />
// 에러! Image 컴포넌트는 반드시 width와 height가 필요합니다

// 해결 1: width, height 직접 지정
<Image src="/photo.jpg" alt="사진" width={400} height={300} />

// 해결 2: fill 속성 사용 (부모 요소 크기에 맞춤)
<div style={{ position: 'relative', width: '400px', height: '300px' }}>
  <Image src="/photo.jpg" alt="사진" fill />
</div>
```

Day 19 정리

개념	핵심 요약
Metadata	`export const metadata` - 정적 SEO 메타데이터 설정
title 템플릿	`template: '%s
generateMetadata	동적 라우트에서 async로 메타데이터 생성
loading.tsx	파일만 만들면 로딩 UI 자동 적용 (React Suspense 기반)
error.tsx	`'use client'` 필수, `reset()` 함수로 다시 시도 기능
not-found.tsx	404 페이지, `notFound()` 함수로 프로그래밍 방식 호출
동적 라우트 `[id]`	URL 파라미터를 대괄호로 표현
next/image	자동 크기 조절, 포맷 변환, Lazy Loading

Day 20 - API Route로 백엔드 만들기

> **Phase 4: Next.js** | 학습일: 20일차

왜 필요한가?

지금까지 데이터를 서버의 메모리(변수)에 저장했습니다. 하지만 실제 앱에서는 **프론트엔드와 백엔드가 API를 통해 통신**합니다. Next.js의 Route Handlers를 사용하면 **같은 프로젝트 안에서 프론트엔드와 백엔드를 모두 만들 수 있습니다**.

별도의 Express 서버를 설치하거나 다른 프로젝트를 만들 필요가 없습니다!

실생활 비유: 식당의 주방과 홀

- **홀(프론트엔드)**: 손님(사용자)이 메뉴를 보고, 주문서를 작성해서 주방에 전달합니다
- **주방(API Route)**: 주문서를 받아서 요리(데이터 처리)를 하고, 완성된 요리(응답)를 홀에 전달합니다

```
홀 (프론트엔드)                    주방 (API Route)

┌──────────────┐                  ┌──────────────┐
│ 주문서 작성   │  ──요청──→      │ 주문 접수     │
│ (fetch 호출) │                  │ (Request)    │
│              │                  │              │
│ 요리 받음    │  ←──응답──       │ 요리 완성     │
│ (Response)   │                  │ (Response)   │
└──────────────┘                  └──────────────┘
```

---

1. Route Handlers (app/api/route.ts)

기본 구조

Route Handlers는 `app/api/` 폴더 안에 `route.ts` 파일을 만들어서 정의합니다.

```
src/app/api/
  todos/
    route.ts          ← GET /api/todos (목록 조회)
                        POST /api/todos (새 할일 추가)
    [id]/
      route.ts        ← GET /api/todos/1 (상세 조회)
                        PUT /api/todos/1 (수정)
                        DELETE /api/todos/1 (삭제)
```

Before (별도 Express 서버가 필요했을 때)

```
# 프론트엔드 프로젝트
my-react-app/
  src/
    App.tsx

# 백엔드 프로젝트 (별도!)
my-express-server/
  src/
    server.ts
    routes/todos.ts
```

After (Next.js에서는 한 프로젝트에서 모두 해결)

```
my-next-app/
  src/
    app/
      page.tsx              # 프론트엔드
      api/todos/route.ts    # 백엔드 (같은 프로젝트!)
```

---

2. HTTP 메서드별 핸들러

HTTP 메서드란?

HTTP 메서드는 "이 요청이 무엇을 하려는 건지" 알려주는 **동사**입니다.

메서드	역할	실생활 비유
GET	데이터 **조회** (읽기)	메뉴판 보기
POST	데이터 **생성** (추가)	새 주문하기
PUT	데이터 **수정** (업데이트)	주문 변경하기
DELETE	데이터 **삭제**	주문 취소하기

GET - 할일 목록 조회

```
// src/app/api/todos/route.ts
import { NextRequest, NextResponse } from 'next/server';
import { getTodos } from '@/lib/todos';

// GET /api/todos - 모든 할일 조회
export async function GET(request: NextRequest) {
  const todos = await getTodos();

  return NextResponse.json(todos);
  // 응답: [{ id: 1, title: "...", completed: false }, ...]
}
```

POST - 새 할일 추가

```
// src/app/api/todos/route.ts (같은 파일에 추가)
import { addTodo } from '@/lib/todos';

// POST /api/todos - 새 할일 추가
export async function POST(request: NextRequest) {
  const body = await request.json();
  // body = { title: "새로운 할일" }

  // 유효성 검사
  if (!body.title || body.title.trim() === '') {
    return NextResponse.json(
      { error: '할일 제목을 입력해주세요' },
      { status: 400 }  // Bad Request
    );
  }

  const newTodo = await addTodo(body.title);

  return NextResponse.json(newTodo, { status: 201 });
  // 201 = Created (새로 생성됨)
}
```

---

3. NextRequest, NextResponse

NextRequest (요청 객체)

클라이언트가 보낸 요청 정보를 담고 있습니다.

```
export async function POST(request: NextRequest) {
  // JSON 본문 읽기
  const body = await request.json();
  // body = { title: "새로운 할일" }

  // URL 정보
  const url = request.url;
  // "http://localhost:3000/api/todos"

  // 쿼리 파라미터
  const searchParams = request.nextUrl.searchParams;
  const filter = searchParams.get('filter');
  // /api/todos?filter=completed → filter = "completed"
}
```

NextResponse (응답 객체)

서버가 클라이언트에게 보내는 응답을 만듭니다.

```
// JSON 응답
return NextResponse.json({ message: '성공' });

// 상태 코드 지정
return NextResponse.json({ data: todo }, { status: 201 });

// 에러 응답
return NextResponse.json(
  { error: '찾을 수 없습니다' },
  { status: 404 }
);
```

---

4. 동적 라우트 파라미터 ([id])

특정 할일 하나를 조회, 수정, 삭제하려면 **어떤 할일인지** 알아야 합니다. URL에 ID를 포함시킵니다.

```ts
// src/app/api/todos/[id]/route.ts
import { NextRequest, NextResponse } from 'next/server';
import { getTodoById, updateTodo, deleteTodo } from '@/lib/todos';

// GET /api/todos/1 - 특정 할일 조회
export async function GET(
  request: NextRequest,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;
  const todo = await getTodoById(Number(id));

  if (!todo) {
    return NextResponse.json(
      { error: '할일을 찾을 수 없습니다' },
      { status: 404 }
    );
  }

  return NextResponse.json(todo);
}

// PUT /api/todos/1 - 특정 할일 수정
export async function PUT(
  request: NextRequest,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;
  const body = await request.json();
  const updatedTodo = await updateTodo(Number(id), body);

  if (!updatedTodo) {
    return NextResponse.json(
      { error: '할일을 찾을 수 없습니다' },
      { status: 404 }
    );
  }

  return NextResponse.json(updatedTodo);
}

// DELETE /api/todos/1 - 특정 할일 삭제
export async function DELETE(
  request: NextRequest,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;
  const deleted = await deleteTodo(Number(id));

  if (!deleted) {
    return NextResponse.json(
      { error: '할일을 찾을 수 없습니다' },
```

```
      { status: 404 }
    );
  }

  return NextResponse.json({ message: '삭제되었습니다' });
}
```

---

5. HTTP 상태 코드

API를 만들 때 적절한 상태 코드를 반환하는 것이 중요합니다.

상태 코드	이름	의미	사용 시점
200	OK	성공	조회 성공, 수정 성공, 삭제 성공
201	Created	생성 성공	새 데이터 추가 성공 (POST)
400	Bad Request	잘못된 요청	필수 데이터 누락, 유효하지 않은 입력
404	Not Found	찾을 수 없음	존재하지 않는 ID로 조회/수정/삭제 시도
500	Internal Server Error	서버 오류	서버에서 예상치 못한 에러 발생

실생활 비유

- **200 OK** = "주문하신 음식 나왔습니다!"
- **201 Created** = "새 메뉴가 등록되었습니다!"
- **400 Bad Request** = "죄송합니다, 주문서를 잘못 작성하셨네요"
- **404 Not Found** = "죄송합니다, 해당 메뉴는 없습니다"
- **500 Server Error** = "죄송합니다, 주방에 문제가 생겼습니다"

---

6. 프론트엔드에서 fetch 연동

완성된 lib/todos.ts (서버 데이터 저장소)

```typescript
// src/lib/todos.ts
import { Todo } from '@/types/todo';

let todos: Todo[] = [
  { id: 1, title: 'Next.js 배우기', completed: false },
  { id: 2, title: 'TypeScript 복습', completed: true },
  { id: 3, title: '포트폴리오 만들기', completed: false },
];

let nextId = 4;

export async function getTodos(): Promise<Todo[]> {
  return [...todos]; // 복사본 반환
}

export async function getTodoById(id: number): Promise<Todo | undefined> {
  return todos.find((todo) => todo.id === id);
}

export async function addTodo(title: string): Promise<Todo> {
  const newTodo: Todo = {
    id: nextId++,
    title,
    completed: false,
  };
  todos.push(newTodo);
  return newTodo;
}

export async function updateTodo(
  id: number,
  updates: Partial<Todo>
): Promise<Todo | null> {
  const index = todos.findIndex((todo) => todo.id === id);
  if (index === -1) return null;

  todos[index] = { ...todos[index], ...updates };
  return todos[index];
}

export async function deleteTodo(id: number): Promise<boolean> {
  const index = todos.findIndex((todo) => todo.id === id);
  if (index === -1) return false;

  todos.splice(index, 1);
  return true;
}
```

클라이언트 컴포넌트에서 API 호출

```
// src/components/TodoApp.tsx
'use client';

import { useState, useEffect } from 'react';
import { Todo } from '@/types/todo';

export default function TodoApp() {
  const [todos, setTodos] = useState<Todo[]>([]);
  const [input, setInput] = useState('');
  const [loading, setLoading] = useState(true);

  // 할일 목록 가져오기 (GET)
  const fetchTodos = async () => {
    try {
      const response = await fetch('/api/todos');
      const data = await response.json();
      setTodos(data);
    } catch (error) {
      console.error('할일 목록을 가져오지 못했습니다:', error);
    } finally {
      setLoading(false);
    }
  };

  // 컴포넌트 마운트 시 할일 목록 로드
  useEffect(() => {
    fetchTodos();
  }, []);

  // 할일 추가 (POST)
  const addTodo = async () => {
    if (!input.trim()) return;

    try {
      const response = await fetch('/api/todos', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ title: input }),
      });

      if (response.ok) {
        const newTodo = await response.json();
        setTodos([...todos, newTodo]);
        setInput('');
      }
    } catch (error) {
      console.error('할일 추가 실패:', error);
    }
  };

  // 할일 완료 토글 (PUT)
  const toggleTodo = async (id: number) => {
```

```
  const todo = todos.find((t) => t.id === id);
  if (!todo) return;

  try {
    const response = await fetch(`/api/todos/${id}`, {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ completed: !todo.completed }),
    });

    if (response.ok) {
      const updatedTodo = await response.json();
      setTodos(
        todos.map((t) => (t.id === id ? updatedTodo : t))
      );
    }
  } catch (error) {
    console.error('할일 수정 실패:', error);
  }
};

// 할일 삭제 (DELETE)
const removeTodo = async (id: number) => {
  try {
    const response = await fetch(`/api/todos/${id}`, {
      method: 'DELETE',
    });

    if (response.ok) {
      setTodos(todos.filter((t) => t.id !== id));
    }
  } catch (error) {
    console.error('할일 삭제 실패:', error);
  }
};

if (loading) return <p>로딩 중...</p>;

return (
  <div style={{ padding: '20px', maxWidth: '600px', margin: '0 auto' }}>
    <h1>할일 관리 앱</h1>

    {/* 할일 추가 폼 */}
    <div style={{ display: 'flex', gap: '8px', marginBottom: '20px' }}>
      <input
        value={input}
        onChange={(e) => setInput(e.target.value)}
        onKeyDown={(e) => e.key === 'Enter' && addTodo()}
        placeholder="할일을 입력하세요"
        style={{
          flex: 1,
          padding: '8px',
          border: '1px solid #ddd',
          borderRadius: '4px',
```

```
        }}
      />
      <button
        onClick={addTodo}
        style={{
          padding: '8px 16px',
          backgroundColor: '#0070f3',
          color: 'white',
          border: 'none',
          borderRadius: '4px',
          cursor: 'pointer',
        }}
      >
        추가
      </button>
    </div>

    {/* 할일 목록 */}
    <ul style={{ listStyle: 'none', padding: 0 }}>
      {todos.map((todo) => (
        <li
          key={todo.id}
          style={{
            display: 'flex',
            alignItems: 'center',
            gap: '8px',
            padding: '8px',
            borderBottom: '1px solid #eee',
          }}
        >
          <input
            type="checkbox"
            checked={todo.completed}
            onChange={() => toggleTodo(todo.id)}
          />
          <span
            style={{
              flex: 1,
              textDecoration: todo.completed ? 'line-through' : 'none',
              color: todo.completed ? '#999' : '#333',
            }}
          >
            {todo.title}
          </span>
          <button
            onClick={() => removeTodo(todo.id)}
            style={{
              padding: '4px 8px',
              backgroundColor: '#ff4444',
              color: 'white',
              border: 'none',
              borderRadius: '4px',
              cursor: 'pointer',
            }}
```

```
            >
              삭제
            </button>
          </li>
        ))}
      </ul>

      {/* 통계 */}
      <div style={{ marginTop: '20px', color: '#666' }}>
        <p>
          전체: {todos.length}개 |
          완료: {todos.filter((t) => t.completed).length}개 |
          미완료: {todos.filter((t) => !t.completed).length}개
        </p>
      </div>
    </div>
  );
}
```

---

7. 풀스택 할일 앱 전체 구조

Day 16~20에서 배운 모든 것을 합치면 다음과 같은 풀스택 구조가 완성됩니다.

```
src/
├── app/
│   ├── layout.tsx              ← 루트 레이아웃 (Navbar, Metadata)
│   ├── page.tsx                ← 메인 페이지 (서버 컴포넌트)
│   ├── loading.tsx             ← 전역 로딩 UI
│   ├── error.tsx               ← 전역 에러 처리 ('use client')
│   ├── not-found.tsx           ← 404 페이지
│   ├── globals.css             ← 전역 스타일
│   ├── todos/
│   │   ├── page.tsx            ← 할일 목록 페이지
│   │   └── [id]/
│   │       └── page.tsx        ← 할일 상세 페이지 (동적 라우트)
│   ├── completed/
│   │   └── page.tsx            ← 완료 목록 페이지
│   ├── stats/
│   │   └── page.tsx            ← 통계 페이지
│   └── api/
│       └── todos/
│           ├── route.ts        ← GET (목록), POST (추가)
│           └── [id]/
│               └── route.ts    ← GET (상세), PUT (수정), DELETE (삭제)
├── components/
│   ├── Navbar.tsx              ← 네비게이션 바 ('use client')
│   ├── TodoApp.tsx             ← 할일 앱 ('use client')
│   └── LoadingSpinner.tsx      ← 로딩 스피너
├── lib/
│   └── todos.ts                ← 서버 데이터 로직
└── types/
    └── todo.ts                 ← 타입 정의
```

데이터 흐름 요약

```
사용자 → 브라우저 (프론트엔드)
           |
           | fetch('/api/todos')
           ▼
       API Route (route.ts)
           |
           | getTodos(), addTodo() 등
           ▼
       lib/todos.ts (데이터 저장소)
           |
           | 결과 반환
           ▼
       API Route → NextResponse.json()
           |
           | JSON 응답
           ▼
       브라우저 → 화면 업데이트
```

자주 하는 실수 모음

실수 1: route.ts 대신 page.tsx로 만들기

```
// 잘못된 구조
src/app/api/todos/page.tsx  ← 이것은 페이지! API가 아닙니다!

// 올바른 구조
src/app/api/todos/route.ts  ← 이것이 API Route입니다
```

실수 2: Content-Type 헤더 빠뜨리기

```
// 잘못된 코드
await fetch('/api/todos', {
  method: 'POST',
  body: JSON.stringify({ title: '새 할일' }),
  // Content-Type 헤더가 없으면 서버가 JSON을 파싱하지 못함!
});

// 올바른 코드
await fetch('/api/todos', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },  // 필수!
  body: JSON.stringify({ title: '새 할일' }),
});
```

실수 3: request.json()에 await 빠뜨리기

```
// 잘못된 코드
export async function POST(request: NextRequest) {
  const body = request.json(); // Promise 객체가 반환됨!
  console.log(body.title);     // undefined
}

// 올바른 코드
export async function POST(request: NextRequest) {
  const body = await request.json(); // await 필수!
  console.log(body.title);           // "새 할일"
}
```

API 테스트 방법

개발 중 API가 제대로 동작하는지 터미널에서 curl 명령어로 테스트할 수 있습니다.

```
# 1. 할일 목록 조회
curl http://localhost:3000/api/todos

# 2. 새 할일 추가
curl -X POST http://localhost:3000/api/todos \
  -H "Content-Type: application/json" \
  -d '{"title": "curl로 추가한 할일"}'

# 3. 특정 할일 조회
curl http://localhost:3000/api/todos/1

# 4. 할일 수정 (완료 처리)
curl -X PUT http://localhost:3000/api/todos/1 \
  -H "Content-Type: application/json" \
  -d '{"completed": true}'

# 5. 할일 삭제
curl -X DELETE http://localhost:3000/api/todos/1
```

Day 20 정리

개념	핵심 요약
Route Handlers	`app/api/todos/route.ts` - API 엔드포인트
HTTP 메서드	GET(조회), POST(생성), PUT(수정), DELETE(삭제)
NextRequest	요청 객체 - `request.json()`, `request.url`
NextResponse	응답 객체 - `NextResponse.json(data, { status })`
동적 라우트	`[id]/route.ts` - `params.id` 로 URL 파라미터 접근
상태 코드	200(OK), 201(Created), 400(Bad Request), 404(Not Found)
fetch 연동	클라이언트에서 `/api/todos` 로 CRUD 요청
풀스택 구조	프론트엔드 + API Route + 데이터 로직이 한 프로젝트에

Phase 4 전체 요약

Day 16부터 Day 20까지 학습한 내용을 정리합니다.

Day	주제	핵심 배운 것
16	React에서 Next.js로	SSR vs CSR, App Router, 'use client', 파일 기반 라우팅
17	라우팅과 레이아웃	Link, usePathname, 레이아웃 중첩, Navbar
18	서버/클라이언트 컴포넌트	async 서버 컴포넌트, 데이터 전달 패턴, 컴포넌트 분리 전략
19	Next.js 세부 기능	Metadata, loading.tsx, error.tsx, not-found.tsx, next/image
20	API Route	Route Handlers, CRUD API, fetch 연동, 풀스택 앱 완성

> **다음 Phase**: Phase 5 - 데이터베이스 (Day 21~23에서 SQL 기본과 데이터 모델링을 배웁니다)